Tugas Besar IF3070 Dasar Inteligensi Artifisial
Kelompok 20 - Pangpang



Anggota:

Haidarrozan Pramasony	18224103
Fikrifalah Muslich	18224069
Made Krisna Kusuma Wijaya	18224091

Daftar Isi

1. Deskripsi Persoalan.....4
1.1. Latar Belakang Masalah.....4
1.2. Variabel Permasalahan.....4
1.3. Ketentuan Representasi State dan Langkah Neighbor.....5
1.4. Ketentuan Representasi State dan Langkah Neighbor.....5
2. Pembahasan.....7
3. Kesimpulan dan Saran.....8
4. Pembagian tugas tiap anggota kelompok.....9
5. Referensi.....11
6. Lampiran Form Penggunaan Generative AI.....12
7. Lampiran.....13

Daftar Tabel

Tabel 4.1 pembagian tugas..... 9

1. Deskripsi Persoalan

1.1. Latar Belakang Masalah

PT Owen Intim Line merupakan perusahaan yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman (shipping fee), dan batas waktu kedatangan (ETA) yang berbeda-beda.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan tata letak kendaraan belum tentu menghasilkan keuntungan yang optimal. Oleh karena itu, dibutuhkan sistem pendukung keputusan berbasis algoritma Local Search untuk menentukan kendaraan mana yang akan diangkut beserta posisi dan orientasinya pada geladak kapal agar memberikan total keuntungan yang maksimal dengan tetap mematuhi seluruh batasan kapasitas dan dimensi geladak.

1.2. Variabel Permasalahan

A. Kendaraan (Vehicle)

- **id**: Identifier unik kendaraan (contoh: "Owen").
- **Dimensions**: Ukuran kendaraan dalam satuan grid yang terdiri atas lebar (width / w) dan panjang (length / l), contoh: {w: 3, l: 4}.
- **Orientation**: Orientasi penempatan kendaraan pada geladak, yaitu Horizontal atau Vertical.
- **ShippingFee**: Tarif atau keuntungan pengiriman kendaraan dalam satuan juta rupiah (contoh: 60).
- **Weight**: Berat kendaraan dalam satuan unit (contoh: 5).
- **ETA**: Estimated Time of Arrival, perkiraan waktu kedatangan kendaraan dalam sisa hari (contoh: 3).

B. Kapal (Ship) dan Armada

- **Dimensions**: Ukuran geladak pada kapal dalam satuan grid yang terdiri atas lebar (width / w) dan panjang (length / l), contoh: {w: 10, l: 20}.

- **MaxCapacity**: Kapasitas angkut berat maksimum suatu kapal dalam satuan unit (contoh: 500).
- **Armada (Spesifikasi Multi-Kapal)**: Kumpulan dari satu atau lebih kapal kargo yang tersedia untuk mengangkut daftar kendaraan guna memaksimalkan total nilai yang diangkut oleh armada.

1.3.Ketentuan Representasi State dan Langkah Neighbor

- **Representasi Grid**: Kendaraan dan geladak kapal direpresentasikan sebagai grid dua dimensi dengan koordinat bilangan bulat.
- **Status Kendaraan**: Kendaraan dapat memiliki ukuran yang berbeda-beda dan tidak semua kendaraan harus dimasukkan ke dalam geladak kapal. Kendaraan yang tidak terangkut berada di luar kapal (seluruh bagiannya berada di luar kapal).
- **Inisialisasi (Initial State)**: Konfigurasi penempatan awal dibangkitkan secara acak (random).
- **Langkah Neighbor (Move) tiap Iterasi**:
    a. Menukar posisi dua kendaraan, baik yang berada di dalam kapal maupun antara kendaraan di dalam dan di luar kapal.
    b. Memindahkan sebuah kendaraan ke koordinat yang berbeda di dalam geladak kapal.
    c. Mengubah orientasi kendaraan (Horizontal menjadi Vertical atau sebaliknya).
- **Representasi Solusi (Final State)**: Menunjukkan keadaan seluruh kendaraan (di dalam maupun di luar kapal), pemetaan koordinat pada geladak kapal, serta nilai objective function dari state tersebut.

1.4.Objective Function dan Constraints

- **Objective Function Wajib**: Memaksimalkan total ShippingFee dari seluruh kendaraan yang berhasil ditempatkan di dalam geladak kapal (atau seluruh armada kapal).
- **Objective Function Tambahan / Heuristik (Bila mengimplementasikan Bonus)**: Mempertimbangkan penalti atau prioritas berdasarkan nilai ETA (ketepatan waktu pengiriman), efisiensi kepadatan ruang grid, atau rasio ShippingFee terhadap Weight dan luas area kendaraan.
- **Constraints (Batasan)**:
    a. **Tidak Boleh Bertumpuk (No Overlapping)**: Tidak boleh ada satu sel grid geladak yang ditempati oleh lebih dari satu kendaraan.

b. **Di Dalam Batas Geladak (Within Boundaries)**: Seluruh sel grid yang ditempati oleh suatu kendaraan harus berada sepenuhnya di dalam dimensi panjang dan lebar geladak kapal.
c. **Batas Kapasitas Berat (Max Capacity)**: Total berat (Weight) dari seluruh kendaraan yang berada di dalam suatu kapal tidak boleh melebihi MaxCapacity kapal tersebut.

2. Pembahasan

2.1. Implementasi Hill Climbing

2.1.1. Gambaran Umum Algoritma

Hill Climbing (HC) merupakan algoritma pencarian lokal (local search) berbasis state tunggal yang bergerak secara iteratif menuju konfigurasi tetangga (neighbor) dengan nilai fungsi objektif yang lebih tinggi. Pada permasalahan penempatan kendaraan di geladak kapal, algoritma ini bertujuan memaksimalkan total ShippingFee dari kendaraan yang berhasil diangkut tanpa melanggar batasan dimensi grid, kapasitas berat maksimum, maupun aturan tidak bertumpuk (no overlapping).

Pada repositori ini, diimplementasikan tiga varian algoritma Hill Climbing di dalam file hillClimbing.py, yaitu:
1. Steepest Ascent Hill-Climbing (ketika parameter max_sideways = 0 dan max_restart = 0).
2. Hill-Climbing with Sideways Move (ketika parameter max_sideways > 0).
3. Random Restart Hill-Climbing (melalui fungsi random_restart() dengan parameter max_restart > 0).

2.1.2. Representasi State dan Inisialisasi Awal

State direpresentasikan oleh kelas Armada dan Ship pada starter.py. Setiap kapal memiliki matriks dua dimensi ship_matrix berukuran w × l yang memetakan setiap sel koordinat geladak ke objek Vehicle yang menempatinya (atau bernilai 0 apabila kosong), serta himpunan list_vehicles_in_ship untuk mencatat kendaraan yang berada di dalam kapal. Kendaraan yang berada di luar kapal ditandai dengan atribut koordinat [-1, -1].

Inisialisasi state awal dilakukan secara acak melalui fungsi random_state_armada() dan random_state_ship(). Fungsi ini memilih kendaraan yang belum ditempatkan secara acak, mengundi orientasi (horizontal atau vertical) serta koordinat awal (x, y) pada geladak, lalu memvalidasinya menggunakan fungsi inputing_vehicle_success().

Proses penempatan awal diulang hingga seluruh kendaraan masuk atau batas percobaan maksimum tercapai.

2.1.3. Pembangkitan Neighbor (Successor State)

Pembangkitan state tetangga dilakukan melalui fungsi get_neighbor(armada). Pada setiap pemanggilan, objek armada disalin secara mendalam (deepcopy) lalu dikenakan salah satu dari tiga operasi langkah (move) yang dipilih secara acak sesuai spesifikasi:
1. Menukar dua kendaraan (switch): Memanggil fungsi switch_vehicle_success() yang memilih dua kendaraan secara acak (baik keduanya di dalam kapal maupun salah satu di luar kapal) dan menukar posisi keduanya apabila tidak melanggar kapasitas berat maupun batas grid.
2. Memindahkan kendaraan (move): Memilih satu kendaraan di dalam kapal secara acak dan memanggil move_vehicle_success() untuk memindahkannya ke koordinat acak baru pada geladak yang sama.
3. Mengubah orientasi kendaraan (rotate): Memilih satu kendaraan di dalam kapal secara acak dan memanggil change_orientation_success() untuk memutar orientasinya antara horizontal dan vertical pada titik koordinat yang sama.

Apabila langkah yang diundi melanggar batasan (terjadi tabrakan sel atau melebihi kapasitas maksimum kapal), mekanisme rollback otomatis mengembalikan kendaraan ke posisi dan orientasi semula.

2.1.4. Varian Algoritma dan Kondisi Penghentian

1. Steepest Ascent Hill-Climbing:
Melalui fungsi hill_climbing(armada, max_sideways=0, n_neighbors=50), algoritma membangkitkan sebanyak n_neighbors kandidat tetangga pada setiap iterasi, mengevaluasi total ShippingFee masing-masing kandidat menggunakan calculate_armada_shipping_fee(), dan menyimpan kandidat dengan nilai tertinggi ke dalam daftar kandidat. Apabila nilai terbaik tersebut lebih besar daripada nilai state saat ini (fee_terbaik > fee_sekarang), algoritma berpindah ke salah satu kandidat terbaik tersebut. Pencarian berhenti ketika tidak ada tetangga yang memiliki nilai lebih tinggi (mencapai puncak lokal).

2. Hill-Climbing with Sideways Move:

Varian ini menggunakan fungsi yang sama namun dengan parameter max_sideways > 0. Ketika tidak ditemukan tetangga dengan nilai yang lebih tinggi tetapi terdapat tetangga dengan nilai objektif yang sama (fee_terbaik == fee_sekarang), algoritma tetap melakukan perpindahan selama jumlah langkah mendatar berturut-turut belum mencapai batas maksimum (sideways < max_sideways). Penghitung sideways di-reset kembali menjadi 0 setiap kali algoritma berhasil menemukan langkah menanjak (fee_terbaik > fee_sekarang). Pencarian dihentikan apabila sideways mencapai max_sideways atau seluruh tetangga bernilai lebih buruk.

3. Random Restart Hill-Climbing:

Melalui fungsi random_restart(armada, max_sideways, n_neighbor, max_restart), algoritma menjalankan pencarian Hill Climbing berulang kali dari titik awal acak yang baru menggunakan fungsi new_random_armada(armada) sebanyak parameter max_restart. Setiap kali satu siklus pencarian berhenti di local optima, nilai akhirnya dibandingkan dengan solusi terbaik sejauh ini (best_fee). Pencarian berhenti setelah jumlah restart mencapai batas maksimum, dan mengembalikan konfigurasi armada terbaik dari seluruh restart.

2.2. Hasil dan Analisis Eksperimen Hill Climbing

2.2.1. Konfigurasi Eksperimen

Eksperimen dilakukan dengan menjalankan masing-masing varian Hill Climbing sebanyak 3 kali pada skenario benchmark yang sama (35 kendaraan, 1 kapal berukuran 10 × 20 grid, dan kapasitas maksimum 200 unit) dengan jumlah sampel tetangga per iterasi n_neighbors = 50. Untuk varian Sideways Move digunakan parameter max_sideways = 20, sedangkan untuk Random Restart digunakan parameter max_restart = 10.

2.2.2. Hasil Eksperimen

Tabel 2.x. Hasil Percobaan Algoritma Hill Climbing

Varian Algoritma	Run	Nilai Objektif Awal	Nilai Objektif Akhir	Banyak Iterasi	Parameter Tambahan / Detail Restart	Durasi (s)
Steepest Ascent HC	1	361	428	6	-	0,0895
Steepest Ascent HC	2	376	379	1	-	0,0296
Steepest Ascent HC	3	375	404	7	-	0,0993
HC with Sideways Move	1	366	419	45	max_sideways = 20	0,5914
HC with Sideways Move	2	394	429	24	max_sideways = 20	0,3496
HC with Sideways Move	3	394	427	55	max_sideways = 20	0,7777
Random Restart HC	1	341	423	25 (total)	10 restart (rata-rata 2,5 iterasi/restart)	0,4620
Random Restart HC	2	395	429	29 (total)	10 restart (rata-rata 2,9 iterasi/restart)	0,5102
Random Restart HC	3	373	424	24 (total)	10 restart (rata-rata 2,4 iterasi/restart)	0,4379

Tabel 2.x. Ringkasan Rata-Rata Hasil Eksperimen Hill Climbing

Varian Algoritma	Rata-rata Objektif Awal	Rata-rata Objektif Akhir	Rata-rata Iterasi	Rata-rata Durasi (s)
Steepest Ascent HC	370,67	403,67	4,67	0,073
HC with Sideways Move	384,67	425,00	41,33	0,573
Random Restart HC	369,67	425,33	26,00	0,470

2.2.3. Analisis Perbandingan Varian Hill Climbing

1. Kualitas Solusi dan Kedekatan terhadap Global Optima:
Steepest Ascent Hill-Climbing menghasilkan rata-rata nilai objektif akhir paling rendah (403,67) dengan variasi hasil yang lebar (379–428). Hal ini terjadi karena algoritma langsung berhenti begitu tidak menemukan langkah yang menaikkan ShippingFee secara langsung; padahal untuk memasukkan atau menukar kendaraan

bernilai tinggi, sering kali dibutuhkan pergeseran posisi atau rotasi kendaraan yang tidak langsung mengubah total ShippingFee.

Sebaliknya, penambahan Sideways Move (rata-rata 425,00) dan Random Restart (rata-rata 425,33) meningkatkan kualitas solusi secara signifikan hingga mencapai sekitar 96,6%–97,5% dari optimum global (440). Sideways Move memberi ruang bagi algoritma untuk menata ulang posisi/orientasi kendaraan pada nilai objektif yang sama (plateau/shoulder) sehingga membuka ruang bagi pertukaran kendaraan bernilai lebih tinggi pada iterasi berikutnya, sedangkan Random Restart mengatasi ketergantungan pada satu state awal yang buruk.

2. Banyak Iterasi dan Durasi Pencarian:

Steepest Ascent berhenti sangat cepat hanya dalam 1 hingga 7 iterasi (rata-rata 0,073 detik) karena cepat terjebak di local optima. Pada Hill-Climbing with Sideways Move, jumlah iterasi meningkat menjadi rata-rata 41,33 iterasi (rata-rata 0,573 detik) karena algoritma terus menelusuri plateau hingga kuota max_sideways = 20 habis di puncak akhir. Pada Random Restart Hill-Climbing dengan 10 restart, setiap restart rata-rata membutuhkan 2,4 hingga 2,9 iterasi dengan total durasi rata-rata 0,470 detik.

3. Konsistensi Hasil Akhir:

Steepest Ascent memiliki konsistensi paling rendah (rentang 379–428, selisih 49 poin) akibat sensitivitas tinggi terhadap konfigurasi acak awal. Hill-Climbing with Sideways Move (rentang 419–429) dan Random Restart Hill-Climbing (rentang 423–429) jauh lebih konsisten karena mampu keluar dari jebakan local optima awal.

2.1. Inisialisasi

A. Representasi State & Struktur Data

Struktur data dibagi menjadi entitas Vehicle untuk merepresentasikan objek kendaraan dan Ship yang menyimpan matriks geladak 2D. Untuk versi bonus multi-kapal, entitas Armada mengelompokkan beberapa kapal.

Python

```
class Vehicle:
        def __init__(self, id: str, w: int, l: int, shipping_fee: float, weight: int, eta: int, coor=None, orientation="vertical"):
        self.id = id
        self.w = w
        self.l = l
        self.orientation = orientation
        self.shipping_fee = shipping_fee
        self.weight = weight
        self.eta = eta
        if coor is None:
            self.coor = [-1, -1]  # [-1, -1] menandakan kendaraan berada di luar kapal
        else:
            self.coor = [coor[0], coor[1]]


class Ship:
    def __init__(self, w: int, l: int, max_capacity: int, id: str, list_vehicles_in_ship=None):
        self.w = w
        self.l = l
        self.max_capacity = max_capacity
        self.ship_matrix = np.full((w, l), 0, dtype=object)  # Grid 2D geladak kapal
        self.id = id
        if list_vehicles_in_ship is None:
            self.list_vehicles_in_ship = set()
        else:
            self.list_vehicles_in_ship = set(list_vehicles_in_ship)
```

B. Validasi Batasan (Constraints)

Sebelum penempatan kendaraan dilakukan, fungsi mengecek tiga batasan utama: tidak keluar batas geladak, tidak tumpang tindih (*no overlapping*), dan total berat tidak melebihi kapasitas maksimum kapal.

Python

```
def is_coor_filled(self, x_coor, y_coor) -> bool:
    # Cek apakah koordinat berada di luar batas geladak atau sudah terisi
    if x_coor >= self.w or x_coor < 0:
        return True
    if y_coor >= self.l or y_coor < 0:
        return True
    if self.ship_matrix[x_coor, y_coor] != 0:
        return True
    return False

def is_space_filled(self, w, l, start_x_coor, start_y_coor):
    # Cek seluruh sel area yang ditempati kendaraan
    for x_coor in range(w):
        for y_coor in range(l):
            if self.is_coor_filled(start_x_coor + x_coor, start_y_coor + y_coor):
                return True
    return False
```

```
def is_ship_full(self, vehicle: Vehicle) -> bool:
    # Cek batasan kapasitas beban maksimum
    return self.calculate_ship_weight() + vehicle.weight > self.max_capacity
```

C. Evaluasi Fungsi Objektif

Fungsi objektif dievaluasi dengan menjumlahkan seluruh shipping_fee kendaraan yang berhasil dimuat ke dalam kapal.

Python

```
def calculate_shipping_fee(self) -> float:
    sum_shipping_fee = 0
    for vehicle in self.list_vehicles_in_ship:
        sum_shipping_fee += vehicle.shipping_fee
    return sum_shipping_fee
```

D. Operasi Neighbor (Move Operator)

Operasi ini berfungsi untuk menghasilkan *state* tetangga dalam *local search*. Jika pergerakan melanggar batasan (*invalid move*), dilakukan mekanisme *rollback* ke posisi semula.

a. Mengubah Orientasi (Rotate)

Python

```
def change_orientation_success(self, vehicle: Vehicle) -> bool:
    if vehicle not in self.list_vehicles_in_ship or vehicle.coor == [-1, -1]:
        return False

    original_orientation = vehicle.orientation
    x_coor, y_coor = vehicle.coor[0], vehicle.coor[1]

    # Lepas sementara kendaraan dari grid
    self.remove_vehicle_success(vehicle)
    vehicle.orientation = "vertical" if vehicle.orientation == "horizontal" else "horizontal"

    # Coba masukkan kembali dengan orientasi baru
    if self.inputing_vehicle_success(vehicle, x_coor, y_coor):
        return True

    # Rollback jika gagal
    vehicle.orientation = original_orientation
    self.inputing_vehicle_success(vehicle, x_coor, y_coor)
    return False
```

b. Memindahkan Posisi / Koordinat (Translate)

Python

```
def move_vehicle_success(self, vehicle: Vehicle) -> bool:
    if vehicle not in self.list_vehicles_in_ship or vehicle.coor == [-1, -1]:
```

```
        return False

    old_x, old_y = vehicle.coor
    new_coor_x = random.randrange(0, self.w)
    new_coor_y = random.randrange(0, self.l)

    self.remove_vehicle_success(vehicle)

    # Coba tempatkan di koordinat baru secara acak
    if self.inputing_vehicle_success(vehicle, new_coor_x, new_coor_y):
        return True

    # Rollback jika koordinat baru tidak valid
    self.inputing_vehicle_success(vehicle, old_x, old_y)
    return False
```

c. Menukar Posisi Dua Kendaraan (Swap)

Python

```
def switch_vehicles_success(self, vehicle_a: Vehicle, vehicle_b: Vehicle) -> bool:
    in_a = vehicle_a in self.list_vehicles_in_ship
    in_b = vehicle_b in self.list_vehicles_in_ship

    if not in_a and not in_b:
        return False

    coor_a = list(vehicle_a.coor)
    coor_b = list(vehicle_b.coor)

    if in_a:
        self.remove_vehicle_success(vehicle_a)
```

```
    if in_b:
        self.remove_vehicle_success(vehicle_b)

    ok_a = (not in_b) or self.inputing_vehicle_success(vehicle_a, *coor_b)
    ok_b = ok_a and ((not in_a) or self.inputing_vehicle_success(vehicle_b, *coor_a))

    if ok_a and ok_b:
        return True

    # Rollback jika penukaran gagal
    if in_b:
        self.remove_vehicle_success(vehicle_a)
    if in_a:
        self.remove_vehicle_success(vehicle_b)
    if in_a:
        self.inputing_vehicle_success(vehicle_a, *coor_a)
    if in_b:
        self.inputing_vehicle_success(vehicle_b, *coor_b)
    return False
```

E. Inisialisasi State Awal (Random Initial State)

Kondisi awal (*initial state*) dibangun secara acak dengan mencoba memasukkan kendaraan ke dalam kapal sampai batas pengulangan maksimum tercapai.

Python

```
def random_state_ship(self, vehicles: list[Vehicle]):
    unplaced_vehicles = list(vehicles)
    attempts = 0
    max_attempts = 100
    options = ["vertical", "horizontal"]
```

```
    while len(unplaced_vehicles) != 0 and attempts < max_attempts:
        attempts += 1
        selected_vehicle = random.choice(unplaced_vehicles)
        selected_vehicle.orientation = random.choice(options)

        random_x = random.randrange(0, self.w)
        random_y = random.randrange(0, self.l)

        if self.inputing_vehicle_success(selected_vehicle, random_x, random_y):
            unplaced_vehicles.remove(selected_vehicle)
            attempts = 0
    return unplaced_vehicles
```

2.2.Simulated Annealing

A. Konstruktor dan Inisialisasi (__init__)

Metode __init__ bertugas menginisialisasi parameter utama algoritma, menyimpan representasi *state* armada, serta menyiapkan atribut pencatatan metrik eksperimen.

Python

```
    class simulated_annealing:
        all_duration = []
        all_stuck_local_frequency = []
        all_shipping_fee_gain = []

        def __init__(self, armada: Armada, T: float, cooling_rate: float):
            self.armada_awal = copy.deepcopy(armada)
            self.armada = copy.deepcopy(armada)
            self.best_armada = copy.deepcopy(armada)
            self.T = T
```

```
        self.cooling_rate = cooling_rate
        self.best_shipping_fee = self.armada.calculate_armada_shipping_fee()

        self.duration = 0.0
        self.stuck_local_frequency = 0
        self.shipping_fee_iteration = []
        self.history_acceptance_rate = []
        self.history_shipping_fee = [self.armada.calculate_armada_shipping_fee()]
```

Penjelasan Atribut:

- **armada_awal, armada, dan best_armada**: Masing-masing menyimpan salinan penuh (deepcopy) dari kondisi awal armada, kondisi armada yang sedang dievaluasi pada iterasi berjalan (*current state*), dan konfigurasi armada dengan nilai *objective function* tertinggi yang pernah ditemukan selama pencarian.
- **T dan cooling_rate**: Menyimpan temperatur awal (T) dan faktor pengali penurunan temperatur (antara 0 dan 1) pada setiap langkah evaluasi.
- **best_shipping_fee**: Menyimpan nilai *shipping fee* terbaik yang diinisialisasi dengan total tarif dari *state* awal.
- **Variabel Pencatatan Metrik**: Atribut stuck_local_frequency, history_acceptance_rate, history_shipping_fee, dan duration digunakan untuk merekam riwayat pencarian di setiap iterasi serta statistik akhir algoritma.

B. Pembangkitan *Neighbor* Acak (find_random_neighbour)

Metode find_random_neighbour berfungsi membangkitkan satu *successor state* (*neighbor*) yang valid secara acak dari *current state*.

Python

```
    def find_random_neighbour(self) -> Armada:
        move_found = False
        neighbour = copy.deepcopy(self.armada)
        max_iteration = 500
        iteration = 0
        while iteration <= max_iteration:
            iteration += 1
```

```
            random_num = random.randint(1, 3)
            match random_num:
                case 1:
                    if neighbour.switch_vehicle_success():
                        move_found = True
                case 2:
                    chosen_ship = random.choice(list(neighbour.ship_list))
                    if len(chosen_ship.list_vehicles_in_ship) > 0:
                                                                chosen_vehicle =
random.choice(list(chosen_ship.list_vehicles_in_ship))
                        if chosen_ship.move_vehicle_success(chosen_vehicle):
                            move_found = True
                case 3:
                    chosen_ship = random.choice(list(neighbour.ship_list))
                    if len(chosen_ship.list_vehicles_in_ship) > 0:
                                                                chosen_vehicle =
random.choice(list(chosen_ship.list_vehicles_in_ship))
                        if chosen_ship.change_orientation_success(chosen_vehicle):
                            move_found = True

            if move_found:
                return neighbour
        return None
```

Penjelasan Mekanisme:

1. Metode membuat salinan (deepcopy) dari self.armada ke dalam variabel neighbour agar modifikasi tidak merusak *state* saat ini apabila percobaan gagal.
2. Di dalam *loop* dengan batas maksimal 500 kali percobaan (max_iteration = 500), algoritma mengundi angka acak antara 1 hingga 3 untuk memilih satu dari tiga operasi *neighbor move* yang diizinkan:
    - **Kasus 1 (switch_vehicle_success)**: Menukar posisi dua kendaraan yang dipilih secara acak, baik antar-kendaraan di dalam kapal maupun antara kendaraan di dalam kapal dengan kendaraan di luar kapal.

- **Kasus 2 (move_vehicle_success)**: Memilih satu kapal dan satu kendaraan di dalamnya secara acak, lalu memindahkannya ke koordinat (x, y) baru yang kosong di dalam kapal tersebut.
- **Kasus 3 (change_orientation_success)**: Memilih satu kapal dan satu kendaraan di dalamnya secara acak, lalu memutar orientasinya dari "vertical" ke "horizontal" atau sebaliknya pada koordinat yang sama.

3. Setiap operasi secara otomatis memvalidasi batasan (*constraints*) melalui metode pada kelas Ship dan Armada, yaitu memastikan kendaraan tidak keluar batas geladak, tidak saling bertumpuk (*no overlapping*), dan tidak melebihi kapasitas beban maksimum kapal.
4. Segera setelah salah satu operasi berhasil (move_found = True), objek neighbour langsung dikembalikan. Jika hingga 500 percobaan tidak ditemukan langkah yang valid, metode mengembalikan None.

C. Evaluasi Perpindahans (move_success)

Metode move_success menentukan apakah *current state* akan berpindah ke neighbor hasil pembangkitan atau tetap bertahan di posisi semula, sekaligus menerapkan jadwal penurunan temperatur (*cooling schedule*).

Python

```
    def move_success(self, neighbor: Armada) -> bool:
        if neighbor is None:
            return False

                                delta = neighbor.calculate_armada_shipping_fee() -
self.armada.calculate_armada_shipping_fee()
        if delta >= 0:
            self.armada = neighbor
            self.T = self.T * self.cooling_rate
            return True
        else:
            prob = math.exp(delta / self.T)
            self.history_acceptance_rate.append(prob)
            if random.random() < prob:
                self.armada = neighbor
                self.T = self.T * self.cooling_rate
                return True
```

```
            else:
                self.stuck_local_frequency += 1
                self.T = self.T * self.cooling_rate
                return False
```

Penjelasan Logika Evaluasi:

1. **Perhitungan Selisih Nilai (delta)**: Dihitung dari selisih nilai *objective function* (*shipping fee*) antara neighbor dengan self.armada saat ini (delta = fee_neighbor - fee_current).
2. **Langkah Membaik atau Setara (delta >= 0)**: Jika neighbor menghasilkan total tarif yang lebih besar atau sama dengan *state* saat ini, perpindahan langsung diterima (self.armada = neighbor), temperatur diturunkan dengan rumus T = T * cooling_rate, dan metode mengembalikan True.
3. **Langkah Memburuk (delta < 0)**: Jika neighbor memiliki nilai yang lebih rendah, algoritma menghitung probabilitas penerimaan menggunakan fungsi eksponensial prob = exp(delta / T). Nilai probabilitas tersebut disimpan ke dalam history_acceptance_rate. Selanjutnya dibangkitkan bilangan acak antara 0 hingga 1 menggunakan random.random():
    - Apabila nilai acak lebih kecil dari prob, maka *state* yang lebih buruk tersebut **diterima** sebagai self.armada, temperatur diturunkan, dan metode mengembalikan True.
    - Apabila nilai acak lebih besar atau sama dengan prob, perpindahan **ditolak**, penghitung frekuensi terjebak di *local optima* (stuck_local_frequency) ditambah 1, temperatur tetap diturunkan, dan metode mengembalikan False.

D. Eksekusi Utama Pencarian (execute)

Metode execute mengatur jalannya siklus iterasi *Simulated Annealing* dari kondisi awal hingga kriteria penghentian (*stopping condition*) terpenuhi.

Python

```
        def execute(self):
            start_time = time.time()
            iteration = 0
            max_iteration = 100
            initial_shipping_fee = self.armada.calculate_armada_shipping_fee()
```

```
        while iteration <= max_iteration and self.T > 0.1:
            iteration += 1
            neighbour = self.find_random_neighbour()
            if self.move_success(neighbour):
                iteration = 0
                fee = self.armada.calculate_armada_shipping_fee()
                self.shipping_fee_iteration.append(fee)
                self.history_shipping_fee.append(fee)
                if fee > self.best_shipping_fee:
                                        self.best_armada, self.best_shipping_fee =
copy.deepcopy(self.armada), fee
            else:

self.history_shipping_fee.append(self.armada.calculate_armada_shipping_fee())

        gain = self.best_shipping_fee - initial_shipping_fee
        simulated_annealing.all_shipping_fee_gain.append(gain)

simulated_annealing.all_stuck_local_frequency.append(self.stuck_local_frequency)
        self.duration = time.time() - start_time
        simulated_annealing.all_duration.append(self.duration)
        return self.best_armada, self.best_shipping_fee, self.history_shipping_fee
```

Penjelasan Alur Eksekusi:

1. **Kriteria Berhenti (*Stopping Condition*)**: Perulangan utama terus berjalan selama dua kondisi terpenuhi, yaitu temperatur masih berada di atas ambang batas minimum (self.T > 0.1) dan jumlah kegagalan perpindahan berturut-turut belum melewati max_iteration (100 kali berturut-turut tanpa ada perpindahan yang diterima).
2. **Siklus Iterasi dan Pelacakan Solusi Terbaik**:
    - Pada setiap iterasi, algoritma memanggil find_random_neighbour() dan mengevaluasinya melalui move_success(neighbour).
    - Jika perpindahan berhasil diterima (move_success bernilai True), penghitung stagnasi iteration di-*reset* kembali menjadi 0. Nilai *shipping fee* terbaru dicatat, lalu dibandingkan terhadap self.best_shipping_fee.

    - Karena *Simulated Annealing* dapat menerima langkah yang menurunkan nilai *objective function*, penyimpanan self.best_armada dan self.best_shipping_fee secara terpisah memastikan solusi paling optimal yang pernah dicapai selama proses eksplorasi tidak hilang ketika algoritma mengambil langkah memburuk di iterasi berikutnya.
3. **Finalisasi dan Pengembalian Hasil**: Setelah keluar dari *loop*, algoritma menghitung selisih peningkatan nilai (*gain*) dari *state* awal, mencatat total durasi eksekusi (self.duration), menyimpan statistik ke variabel kelas, dan mengembalikan pasangan solusi terbaik (best_armada), nilai maksimum (best_shipping_fee), serta riwayat nilai tiap iterasi (history_shipping_fee).

Hasil:

[Run 1]

- Initial Shipping Fee : 518
- Final/Best Fee : 733
- Total Iterasi : 180
- Frekuensi Stuck : 22 kali
- Durasi Pencarian : 0.1998 detik

[Run 2]

- Initial Shipping Fee : 518
- Final/Best Fee : 727
- Total Iterasi : 180
- Frekuensi Stuck : 20 kali
- Durasi Pencarian : 0.2109 detik

[Run 3]

- Initial Shipping Fee : 518
- Final/Best Fee : 735
- Total Iterasi : 180

- Frekuensi Stuck : 28 kali

- Durasi Pencarian : 0.1993 detik

2.3. Implementasi Genetic Algorithm

2.3.1. Gambaran Umum Algoritma

Genetic Algorithm (GA) merupakan algoritma pencarian berbasis populasi yang menggunakan prinsip evolusi untuk memperoleh solusi dengan nilai fitness yang semakin baik. Pada permasalahan ini, GA digunakan untuk mengoptimalkan penempatan kendaraan pada geladak kapal dengan memaksimalkan total ShippingFee, tanpa melanggar batasan dimensi, kapasitas berat, maupun ketentuan bahwa kendaraan tidak boleh saling bertumpuk.

Implementasi GA menggunakan beberapa operator utama, yaitu selection, crossover, mutation, dan elitism. Selain itu, diterapkan heuristik pengisian ruang kosong serta mutasi adaptif untuk meningkatkan efektivitas pencarian solusi.

2.3.2. Representasi Kromosom dan Inisialisasi Populasi

Setiap individu dalam populasi direpresentasikan menggunakan kelas Individual, sedangkan kromosom disimpan dalam dictionary genes. Setiap kendaraan memiliki satu gen yang menyimpan informasi ship_id, is_loaded, koordinat x dan y, serta orientation.

Representasi ini memungkinkan algoritma menentukan kapal tujuan, posisi kendaraan, orientasi penempatan, dan status apakah kendaraan berhasil dimuat. Kendaraan yang tidak dimuat tetap direpresentasikan dalam kromosom sehingga dapat dipertimbangkan kembali pada proses evolusi berikutnya.

Populasi awal dibangkitkan melalui fungsi randomize() dengan mengacak urutan kendaraan, orientasi, kapal tujuan, dan koordinat penempatan. Setiap penempatan diperiksa agar tidak melanggar batasan. Dalam penggunaan melalui GUI, konfigurasi armada yang sudah tersedia juga dimasukkan sebagai salah satu individu awal.

2.3.3. Fitness Function dan Validasi Solusi

Fitness function digunakan untuk mengukur kualitas setiap individu berdasarkan total ShippingFee kendaraan yang berhasil ditempatkan secara valid. Semakin besar nilai fitness, semakin baik solusi yang dihasilkan.

Fungsi fitness dirumuskan sebagai berikut:

$$F(S) = \Sigma\ (f_i \times z_i), \text{ untuk } i = 1 \text{ hingga } n$$

Dengan:

- F(S) = nilai fitness dari solusi S.

- f_i = ShippingFee kendaraan ke-i.
- z_i = 1 apabila kendaraan ke-i berhasil dimuat secara valid, dan 0 apabila tidak dimuat.
- n = jumlah seluruh kendaraan.

Evaluasi dilakukan melalui fungsi evaluate(). Kandidat kendaraan diurutkan berdasarkan ShippingFee secara menurun, kemudian diperiksa terhadap batas geladak, kapasitas berat, dan kemungkinan overlap. Kendaraan yang tidak memenuhi salah satu batasan dinyatakan tidak dimuat dan tidak berkontribusi terhadap fitness.

Dengan mekanisme tersebut, fitness hanya menghitung kendaraan yang memenuhi seluruh constraints. Variabel ETA tidak digunakan dalam perhitungan fitness pada implementasi ini.

2.3.4. Selection dan Crossover

Proses pemilihan induk menggunakan tournament selection melalui fungsi tournament_selection() dengan ukuran turnamen sebesar tiga individu. Tiga kandidat dipilih secara acak dari populasi, kemudian individu dengan fitness tertinggi dipilih sebagai induk.

Crossover dilakukan menggunakan fungsi crossover() dengan probabilitas 0,8. Operator ini menggabungkan gen dari dua induk untuk membentuk satu keturunan. Pemilihan gen dilakukan secara acak, tetapi ketika suatu kendaraan berhasil dimuat pada salah satu induk dan tidak pada induk lainnya, gen dari induk yang berhasil memuat kendaraan memperoleh probabilitas pemilihan sebesar 70%.

Strategi tersebut bertujuan mempertahankan konfigurasi penempatan yang telah berhasil, sekaligus memungkinkan terbentuknya kombinasi solusi baru. Apabila crossover tidak dilakukan, keturunan merupakan salinan induk pertama sebelum memasuki tahap mutasi.

2.3.5. Mutation dan Elitism

Mutasi dilakukan melalui fungsi mutate() dengan probabilitas dasar 0,3. Terdapat empat jenis perubahan gen yang dapat dilakukan, yaitu perubahan koordinat kendaraan, perubahan orientasi, penggantian kendaraan yang sudah dimuat dengan kendaraan di luar kapal yang memiliki ShippingFee lebih tinggi, serta perpindahan kendaraan ke kapal lain pada konfigurasi multi-kapal.

Implementasi juga menggunakan mutasi adaptif. Apabila fitness terbaik tidak mengalami peningkatan selama tiga generasi berturut-turut, probabilitas mutasi dinaikkan menjadi 0,48 dan jumlah perubahan gen dalam satu proses mutasi ditingkatkan dari rentang 1–3 menjadi 2–4 gen. Mekanisme ini bertujuan meningkatkan eksplorasi ketika populasi mengalami stagnasi.

Elitism diterapkan melalui fungsi get_elites() dengan mempertahankan dua individu terbaik secara langsung pada generasi berikutnya. Dengan demikian, solusi terbaik yang telah ditemukan tidak hilang akibat operasi crossover atau mutation.

2.3.6. Heuristik Pengisian Ruang Kosong

Selain operator genetik, implementasi menggunakan fungsi fill_empty_slots() untuk menempatkan kembali kendaraan yang belum berhasil dimuat. Kendaraan tersebut diurutkan

berdasarkan ShippingFee tertinggi, kemudian algoritma mencari kapal dan koordinat yang masih memenuhi batasan.

Pencarian posisi dilakukan melalui kombinasi percobaan koordinat acak dan pemeriksaan grid secara terstruktur menggunakan pendekatan first-fit. Setelah proses penyisipan, fitness individu dievaluasi kembali.

Heuristik ini membantu memanfaatkan ruang geladak yang belum terisi tanpa harus bergantung sepenuhnya pada perubahan gen acak. Pendekatan tersebut menjadikan implementasi GA memiliki komponen perbaikan solusi berbasis heuristik.

2.3.7. Alur Eksekusi dan Kondisi Penghentian

Eksekusi utama dilakukan melalui fungsi run_genetic_algorithm(). Algoritma membangkitkan populasi awal, mengevaluasi fitness individu, dan mengurutkannya berdasarkan fitness tertinggi.

Pada setiap generasi, algoritma mencatat fitness maksimum dan rata-rata populasi. Selanjutnya, dua individu terbaik dipertahankan melalui elitism, sedangkan individu lainnya dihasilkan melalui tournament selection, crossover, mutation, dan perbaikan ruang kosong.

Proses tersebut diulang hingga jumlah generasi yang ditentukan tercapai. Hasil akhirnya berupa individu dengan fitness terbaik beserta konfigurasi penempatan kendaraan, durasi pencarian, dan riwayat fitness setiap generasi.

2.4. Hasil dan Analisis Eksperimen Genetic Algorithm

2.4.1. Konfigurasi Eksperimen

Eksperimen dilakukan untuk mengamati pengaruh jumlah generasi dan ukuran populasi terhadap kualitas solusi serta durasi pencarian Genetic Algorithm.

Seluruh percobaan menggunakan data benchmark yang terdiri atas 35 kendaraan dan satu kapal berukuran 10 × 20 grid dengan kapasitas berat maksimum 200 unit. Crossover rate ditetapkan sebesar 0,8, mutation rate sebesar 0,3, dan jumlah elite sebanyak dua individu.

Eksperimen dibagi menjadi dua kelompok. Pada eksperimen pertama, jumlah populasi dipertahankan sebanyak 50 individu dengan variasi generasi 50, 100, dan 200. Pada eksperimen kedua, jumlah generasi dipertahankan sebanyak 100 dengan variasi populasi 20, 50, dan 100 individu.

Setiap konfigurasi dijalankan tiga kali secara independen dengan inisialisasi populasi awal acak, sehingga diperoleh total 18 percobaan.

2.4.2. Hasil Eksperimen

Tabel 2.x. Hasil 18 Percobaan Genetic Algorithm

Eksperimen	Populasi	Generasi	Run	Fitness awal	Fitness akhir	Durasi (s)
1	50	50	1	425	435	0,5102

Eksperimen	Populasi	Generasi	Run	Fitness awal	Fitness akhir	Durasi (s)
1	50	50	2	427	437	0,4000
1	50	50	3	424	437	0,4653
1	50	100	1	428	437	2,7513
1	50	100	2	429	438	3,1938
1	50	100	3	427	437	2,9940
1	50	200	1	425	436	1,8131
1	50	200	2	433	438	3,1466
1	50	200	3	427	438	1,5244
2	20	100	1	428	437	0,3238
2	20	100	2	423	432	0,2902
2	20	100	3	428	434	0,3376
2	50	100	1	426	436	0,8664
2	50	100	2	426	436	0,7487
2	50	100	3	425	436	0,9612
2	100	100	1	429	438	1,7820
2	100	100	2	426	438	1,6973
2	100	100	3	427	438	1,6478

Untuk mempermudah perbandingan antarkonfigurasi, dihitung rata-rata fitness awal, rata-rata fitness akhir, dan rata-rata durasi dari ketiga percobaan pada setiap konfigurasi.

Tabel 2.x. Ringkasan Rata-Rata Hasil Eksperimen GA

Eksperimen	Populasi	Generasi	Rata-rata fitness awal	Rata-rata fitness akhir	Rata-rata durasi (s)
1	50	50	425,33	436,33	0,459
1	50	100	428,00	437,33	2,980
1	50	200	428,33	437,33	2,161
2	20	100	426,33	434,33	0,317
2	50	100	425,67	436,00	0,859
2	100	100	427,33	438,00	1,709

2.4.3. Analisis Pengaruh Jumlah Generasi

Berdasarkan eksperimen pertama (populasi tetap 50), peningkatan jumlah generasi dari 50 menjadi 100 meningkatkan rata-rata fitness akhir dari 436,33 menjadi 437,33 serta berhasil mencapai nilai fitness tertinggi 438 pada salah satu percobaan. Pada konfigurasi 200 generasi,

rata-rata fitness akhir bertahan di 437,33, tetapi frekuensi pencapaian nilai tertinggi 438 meningkat menjadi dua dari tiga percobaan (Run 2 dan Run 3).

Peningkatan tersebut menunjukkan bahwa tambahan generasi memberikan kesempatan lebih besar bagi algoritma untuk keluar dari stagnasi dan menemukan kombinasi penempatan kendaraan bernilai 438. Namun, selisih kenaikan rata-rata fitness dari 100 ke 200 generasi mulai melandai karena solusi sudah sangat dekat dengan batas kapasitas kapal.

Dari sisi waktu komputasi, durasi eksekusi meningkat dari rata-rata 0,459 detik pada 50 generasi menjadi 2,980 detik pada 100 generasi dan 2,161 detik pada 200 generasi. Fluktuasi durasi antara 100 dan 200 generasi dipengaruhi oleh beban kerja fungsi fill_empty_slots() serta pemicuan mutasi adaptif saat populasi mengalami stagnasi. Secara keseluruhan, peningkatan generasi di atas 100 mulai menunjukkan fenomena diminishing returns terhadap kenaikan rata-rata fitness.

2.4.4. Analisis Pengaruh Ukuran Populasi

Pada eksperimen kedua (generasi tetap 100), peningkatan ukuran populasi memberikan dampak yang jelas dan konsisten terhadap kualitas solusi. Rata-rata fitness akhir meningkat dari 434,33 pada populasi 20, menjadi 436,00 pada populasi 50, dan mencapai 438,00 pada populasi 100.

Pada populasi 20, keterbatasan keragaman genetik awal menyebabkan algoritma lebih rentan terjebak pada local optima (dengan fitness akhir terendah 432 pada Run 2). Sebaliknya, pada populasi 100, besarnya variasi individu pada setiap generasi membuat algoritma berhasil mencapai fitness 438 secara konsisten pada ketiga percobaan.

Rata-rata durasi pencarian meningkat secara proporsional terhadap ukuran populasi, yaitu dari 0,317 detik (populasi 20), menjadi 0,859 detik (populasi 50), hingga 1,709 detik (populasi 100). Berdasarkan hasil ini, populasi 100 memberikan kualitas solusi dan stabilitas tertinggi, sedangkan populasi 50 menjadi kompromi moderat apabila waktu komputasi sub-detik (di bawah 1 detik) diprioritaskan.

2.4.5. Analisis Konvergensi dan Konsistensi

Selama proses evolusi dari generasi awal hingga akhir, seluruh 18 percobaan menunjukkan peningkatan yang signifikan dari fitness awal (berkisar 423–433) menuju fitness akhir (berkisar 432–438). Penggunaan elitism sebanyak dua individu memastikan bahwa nilai fitness maksimum populasi bersifat monoton tidak turun antargenerasi, sementara operator crossover berbias dan heuristik fill_empty_slots() menjaga rata-rata fitness populasi tetap tinggi.

Dari segi konsistensi antarrun, ukuran populasi terbukti sangat memengaruhi stabilitas hasil akhir. Pada eksperimen kedua, rentang fitness akhir menyempit drastis dari 432–437 pada populasi 20 (selisih 5 poin) menjadi 436 pada seluruh percobaan populasi 50, dan 438 pada seluruh percobaan populasi 100. Pada eksperimen pertama dengan populasi 50, seluruh percobaan untuk generasi 50, 100, dan 200 juga stabil berada pada rentang sempit 435–438.

Hasil tersebut membuktikan bahwa semakin besar ruang sampel populasi dan jumlah generasi yang diberikan, semakin konsisten pula Genetic Algorithm dalam mengonvergensi populasi ke nilai yang sama.

2.4.6. Evaluasi Kualitas Solusi terhadap Optimum Global

Untuk benchmark satu kapal yang digunakan, dilakukan verifikasi tambahan menggunakan batas atas persoalan 0/1 Knapsack dengan kapasitas berat 200 unit. Perhitungan tersebut menghasilkan nilai maksimum ShippingFee sebesar 440.

Nilai 440 juga dapat direalisasikan melalui penempatan 12 kendaraan secara valid pada grid kapal berukuran 10 × 20 tanpa overlap dan dengan total berat tepat 200 unit. Karena batas atas dan solusi feasible memiliki nilai yang sama, optimum global benchmark tersebut adalah 440.

Fitness tertinggi yang ditemukan GA pada 18 percobaan adalah 438 (dicapai pada 6 dari 18 percobaan, termasuk 100% percobaan pada konfigurasi Populasi 100, Generasi 100), atau sekitar 99,55% dari optimum global, dengan selisih hanya sebesar 2 satuan ShippingFee. Sementara itu, rata-rata keseluruhan fitness akhir dari 18 percobaan adalah 436,56 (99,22% dari optimum global).

Hasil ini menunjukkan bahwa GA mampu menemukan solusi yang sangat dekat dengan optimum global, meskipun sifat stokastik algoritma dan kerapatan penempatan 2D membuat kombinasi bernilai 440 memiliki ruang konfigurasi yang sangat sempit untuk dicapai.

Verifikasi optimum global ini merupakan analisis tambahan terhadap data benchmark, bukan bagian dari fungsi optimasi GA utama.

2.4.7. Kesimpulan Eksperimen

Berdasarkan 18 percobaan, Genetic Algorithm mampu meningkatkan fitness dari populasi awal secara konsisten dan menghasilkan solusi yang mencapai 99,55% dari optimum global (438 dari 440) dengan tetap memenuhi seluruh constraints penempatan kendaraan.

Peningkatan jumlah generasi dari 50 ke 100–200 meningkatkan peluang tercapainya nilai 438, sedangkan peningkatan ukuran populasi dari 20 ke 50 dan 100 secara langsung meningkatkan rata-rata fitness akhir (434,33 menjadi 436,00 dan 438,00) sekaligus menghilangkan variansi hasil antarrun. Pada benchmark ini, konfigurasi populasi 100 dengan 100 generasi memberikan kualitas dan konsistensi terbaik (fitness 438 pada seluruh run dengan rata-rata durasi 1,709 detik).

Penggunaan elitism, mutasi adaptif, dan heuristik pengisian ruang kosong terbukti efektif membantu mempertahankan serta memperbaiki solusi selama proses evolusi.

3. Kesimpulan dan Saran

3. Kesimpulan dan Saran

3.1. Kesimpulan

1. Permasalahan optimasi penempatan kendaraan pada geladak kapal kargo dapat dimodelkan sebagai persoalan pencarian lokal (local search) pada grid dua dimensi dengan fungsi objektif memaksimalkan total ShippingFee serta batasan dimensi geladak, tidak adanya tumpang tindih (no overlapping), dan kapasitas berat maksimum kapal.

2. Algoritma Hill Climbing (Steepest Ascent, Sideways Move, dan Random Restart) memiliki waktu eksekusi per iterasi yang paling cepat karena hanya mengevaluasi tetangga langsung dari satu state tunggal. Namun, varian dasar Hill Climbing sangat rentan terjebak pada local optima ketika geladak kapal mulai padat. Penambahan mekanisme Sideways Move dan Random Restart membantu algoritma keluar dari flat local maxima dan mengeksplorasi titik awal yang berbeda.

3. Algoritma Simulated Annealing mampu mengurangi risiko terjebak secara prematur pada local optima melalui penerimaan langkah yang memiliki nilai objektif lebih rendah secara probabilistik berdasarkan parameter suhu (T) dan cooling rate. Seiring menurunnya suhu, nilai probabilitas penerimaan e^(ΔE/T) ikut menurun sehingga pencarian bertransisi secara bertahap dari eksplorasi menuju eksploitasi.

4. Algoritma Genetic Algorithm memberikan hasil pencarian yang paling mendekati optimum global dan paling konsisten dibandingkan algoritma berbasis state tunggal karena memelihara keragaman solusi dalam satu populasi sekaligus. Kombinasi tournament selection, biased crossover, mutasi adaptif, elitism, serta heuristik pengisian ruang kosong (fill_empty_slots) mampu mencapai fitness akhir hingga 438 dari optimum global 440 (99,55%).

5. Pada eksperimen Genetic Algorithm, peningkatan ukuran populasi (dari 20 menjadi 50 dan 100) serta jumlah generasi (dari 50 menjadi 100 dan 200) meningkatkan rata-rata fitness akhir dan mempersempit variansi hasil antarrun, tetapi diiringi peningkatan durasi komputasi secara proporsional serta fenomena diminishing returns ketika solusi sudah mendekati kapasitas maksimum kapal.

3.2. Saran

1. Memperluas operator neighbor pada Hill Climbing dan Simulated Annealing agar tidak hanya menukar kendaraan satu-lawan-satu atau menggeser kendaraan di dalam kapal, tetapi juga memungkinkan penyisipan langsung kendaraan dari luar kapal ke ruang kosong geladak yang tercipta setelah proses pergeseran atau rotasi.

2. Menambahkan pemeriksaan kesuksesan langkah (valid move) dan pencegahan state berulang (tabu list sederhana atau hash state) pada Hill Climbing with Sideways Move agar kuota sideways move benar-benar digunakan untuk menjelajahi konfigurasi tata letak baru.

3. Mengintegrasikan fungsi objektif tambahan yang mempertimbangkan variabel ETA (Estimated Time of Arrival) serta efisiensi kepadatan ruang (packing density) agar kendaraan dengan tenggat waktu lebih dekat diprioritaskan dan penataan di dalam kapal menjadi lebih rapat.

4. Melakukan pengujian lanjutan pada skala multi-kapal (armada) dengan ukuran grid dan jumlah kendaraan yang lebih beragam untuk mengukur skalabilitas waktu komputasi ketiga algoritma pada skenario operasional yang lebih besar.

4. Pembagian tugas tiap anggota kelompok

Tabel 4.1 pembagian tugas

Nama	NIM	Tugas
Fikrifalah Muslich	18224069	- Implementasi bagian awalan: pemodelan kelas dan logika dasar Ship, Armada, dan Vehicles (representasi state, validasi batasan grid/kapasitas, dan objective function). - Implementasi algoritma Simulated Annealing beserta pencatatan metrik eksperimen (exp(Delta_E / T) dan frekuensi stuck di local optima). - Penyusunan laporan dan analisis bagian terkait.
Made Krisna Kusuma Wijaya	18224091	- Implementasi bagian awalan: pemodelan kelas dan logika dasar Ship, Armada, dan Vehicles (inisiator state acak dan operasi neighbor move). - Implementasi algoritma Hill-Climbing beserta variannya (Steepest Ascent, Stochastic, Sideways Move, dan Random Restart). - Penyusunan laporan dan analisis bagian terkait.
Haidarrozan Pramasony	18224103	- Implementasi algoritma Genetic Algorithm (representasi kromosom populasi, fitness function, selection, crossover, dan mutation). - Pelaksanaan eksperimen variasi ukuran populasi dan jumlah iterasi pada Genetic Algorithm.

		- Penyusunan laporan dan analisis bagian terkait.

5. Referensi

Materi pada edunex

6. Lampiran Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Haidarrozan Pramasony/18224103, Fikrifalah Muslich/18224069, Made Krisna Kusuma Wijaya/18224091
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatikas
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Intelegensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22
Oktober 2026

Haidarrozan Pramasony / 18224103



Fikrifalah Muslich / 18224069



Made Krisna Kusuma Wijaya



7. Lampiran